

vanilla-extract

Usage, Methods and Validation Results

Version 0.2.0 | verification run 2026-09-16 | MIT licensed | github.com/brandonsbutler-ai/vanilla-extract

This document exists so you do not have to take the claims on faith. It states what the tool does, how it is checked, what the checks measured, and -- the part most such documents omit -- what the numbers do **not** prove.

Every figure below is produced by a script in this repository. You can run them yourself; the commands are at the end.

1. What it is

vanilla-extract extracts text and named fields from documents, and produces a reviewable table from a folder of them. It reads PDF, DOCX, PPTX, XLSX, ODT, RTF, EML, MBOX, HTML, XML, CSV, TSV, JSON and plain text, plus ZIP archives containing any of those.

It has **no third-party dependencies**. Not "few" -- none. That is not a stylistic preference: it means the tool installs in an environment where adding a dependency requires an approval process, and it means there is no supply chain under it to audit.

2. Usage

Read one document

```
vanilla contract.pdf
```

A folder into a spreadsheet

```
vanilla --batch invoices/ --csv results.csv --exceptions skipped.csv
```

Two tables, always. `results.csv` has a row per document read. `skipped.csv` names every document that could **not** be read and why. The second file is the important one, and section 5 explains why.

Pull named fields

```
vanilla --batch invoices/ --csv results.csv \
  --field "invoice_no=Invoice\s*#\s*([A-Z0-9-]+)" \
  --field "total=Total\s*:\s*\$?([0-9,]+\.[0-9]{2})"
```

Or let it find the fields itself

```
vanilla --batch invoices/ --recognize --report review.html --csv results.csv
```

`--recognize` reads label/value pairs out of the documents and ranks them by how many documents carry each label, so the columns come from your corpus rather than from a guess. Each column is typed -- money, date, email, phone, percent, identifier -- by a conservative detector.

Measured on a four-document sample, with no pattern written by hand:

claim number	4 docs (100%)	identifier	e.g. CL-4401
settlement date	4 docs (100%)	date_iso	e.g. 2026-08-02
insurer	4 docs (100%)	text	e.g. Ridgeway Mutual
terms	4 docs (100%)	text	e.g. Net 30
amount	4 docs (100%)	money	e.g. \$2,145.00
contact	4 docs (100%)	email	e.g. claims@ridgeway.example

Review and correct before you trust it

--report review.html writes a single HTML file -- no server, no external assets, nothing uploaded. Every value is editable in place, each document's source text is one click away so a suspect value can be checked against the page it came from, the exceptions table is shown alongside rather than buried, and a button exports the corrected table as CSV. Because nothing leaves the machine, a review of your own confidential documents stays on your machine.

The datasheet: what each file was when it arrived

```
vanilla --batch invoices/ --datasheet state.html --csv out.csv
```

A searchable, sortable page recording each file's state as found: size, modified, accessed and created times, rwx permissions and octal mode, owner and group, symlink status and target, hard-link count, inode, filesystem type, and the extraction outcome. A file that could not be read still appears, with its state and the reason.

Three caveats are built into the output rather than left to the reader:

- **Creation time is not universally available.** Windows records it, macOS and the BSDs expose it, and on Linux it exists only on some filesystems. Every row states where its value came from. The inode-change time is reported under its own name and never presented as creation.
- ****NTFS, exFAT and network mounts report ownership and permissions from mount options, not from the files.**** Those rows are marked not reliable, and the page says why.
- **A file inside a ZIP carries its own timestamp and mode**, which can predate the archive by years. Those come from the archive's central directory.

Keep the original and every correction

```
vanilla --batch invoices/ --recognize --workspace case01/ --report review.html --csv out.csv
vanilla --workspace case01/ --import-csv corrected.csv
vanilla --workspace case01/ --verify
```

```
case01/
  manifest.json      run metadata and a SHA-256 for every artefact
  originals/         your documents as received
  extracted/          what the tool read, per document, never edited
  revisions/          each corrected table, appended, never replaced
```

Filing a correction states exactly what moved:

```
revision 2: 4 rows, 2 cell(s) changed from the previous revision
  INV-1001.txt: customer: 'Northwind Traders' -> 'Northwind Traders LLC'
  INV-1002.txt: total: '$433.00' -> '$433.50'
```

This answers a question that otherwise has no answer three months later: was this value extracted from the document, or typed by a person?

3. How it is verified

Three layers, because they catch different things.

Layer	What it is	What it catches
Unit tests (104)	python3 -m unittest discover -s tests	Logic errors in one function, and every past bug as a regression test
Benchmark	python3 benchmark.py <dir>	Extraction *quality*, scored against an independent implementation
End-to-end verification (176 checks)	python3 verify_e2e.py	Whether the product does what its documentation says

The third layer is the unusual one. Unit tests check units; they cannot tell you the README is wrong. verify_e2e.py generates a fresh corpus in every supported format, drives the real command-line tool as a user would, and asserts each documented claim against actual output. It prints PASS or FAIL per claim with the evidence and exits non-zero on any

failure.

It is built to be able to say the product does not work. That is the only reason it is worth running -- and it has done so twice, which section 7 covers.

4. Extraction quality, measured

PDF output is scored against [poppler](https://poppler.freedesktop.org/)'s `pdftotext`, a mature C++ implementation with full font handling, by **token recall**: the fraction of the reference's words that vanilla_extract also produced, counted as a multiset so repetition is not rewarded.

poppler is the reference, not the target. It is allowed to win; the point is knowing by how much.

Corpus A -- 210 tool-generated PDFs (fpdf2, LibreOffice Writer):

```
token recall : mean 1.000  median 1.000  min 1.000
>= 0.99      : 210/210 (100.0%)
```

Corpus B -- 86 commercial PDFs (Adobe PDF Library, Mac Quartz; illustrated, multi-column, 100-578 pages):

```
files measured : 53
token recall   : mean 0.873  median 0.912  min 0.007  max 0.961
>= 0.95        : 4/53   (7.5%)
>= 0.90        : 35/53  (66.0%)
>= 0.50        : 51/53  (96.2%)

encrypted, refused with a clear error: 33 (reported separately, not scored)
```

Read plainly: **excellent on PDFs produced by software, good but not perfect on complex commercial ones, and never quite matching poppler on the hardest.** If you need the last few percent on illustrated multi-column layouts, use poppler. If you need no dependencies, or you need the field recognition and the audit trail, use this.

Throughput

Over 14 unencrypted PDFs of 5 MB and up:

	pages/second
Median	101
Fastest	179
Slowest measured (image-heavy, four runs)	42

So: **40 pages per second or better, typically around 100, on an otherwise idle machine.** Throughput is wall-clock and measures the machine as much as the code -- the same files drop to 28-35 pages/s at a load average of 4 -- so the verifier reports these numbers alongside the load average that produced them rather than asserting a floor that a busy server would fail for the wrong reason.

A file is read fully into memory, so peak usage tracks document size. One PDF content stream is inflated with a 64 MB ceiling.

5. It fails loudly

The worst behaviour a text extractor can have is returning something that looks like a result. An empty string reads as an empty document. Mojibake poisons a dataset silently. Both get a specific, named error instead:

Condition	Behaviour
Encrypted PDF	EncryptedPDF. Commercial PDFs are routinely encrypted with an <i>*empty*</i> user password purely to set permission flags -- readable in a viewer, still encrypted on disk. 33 of the 86 files in corpus B are like this.
Fonts with no character map	UndecodableText, raised only when <i>*nothing*</i> survives. A page mixing an unmapped decorative heading with readable body text keeps the body.
Unrecognised format	UnsupportedFormat
Decompression bomb	ArchiveTooLarge, raised before allocating

Read fine, contains no text Reported in the exceptions table as `no_text_found` -- almost always a scan with no text layer. Not a blank row.

In batch mode none of these abort the run. Each becomes a row in the exceptions table, so a folder of 400 documents with 3 bad ones yields 397 results and 3 named failures.

6. Hostile input

The tool's entire job is reading files supplied by someone else, so the input is untrusted by definition. These were tested, not assumed:

Attack	Result
ZIP decompression bomb	Refused. Members are vetted against declared size and compression ratio before a byte is read, with a running budget for the archive. A 199 KB file declaring 200 MB is refused in 0.000 s.
PDF decompression bomb	Refused. Inflation is bounded; a 597 KB file declaring 600 MB peaks near 128 MB instead of 616 MB.
XXE (external entity)	Not exploitable. Verified against <code>file:///etc/passwd</code> .
Billion laughs	Bounded by the XML parser a few levels in. Amplification, not denial of service.
Catastrophic backtracking	Three patterns were quadratic and are fixed. See <i>*What an audit of the patterns found*</i> below. Every regex in the package is now measured for growth rather than asserted to be linear.
Spreadsheet formula injection	Neutralized. A document containing <code>=cmd ' /C calc!A0</code> would otherwise execute when the client opened the CSV. Cells leading with <code>= + - @</code> tab or CR are prefixed, in both the file writer and the report's in-browser export.
Script injection into the report	Fixed. <code></script></code> inside a document's text closed the report's data block. Now escaped.
Path traversal	A source label containing <code>../</code> or an absolute path cannot escape a workspace.

Limits are generous on purpose. A real 300-page report is large and legitimate; the point is to refuse the absurd, not the merely big -- a 20,000-paragraph document at a 29:1 compression ratio still extracts in 0.02 s.

7. What the verification found

A validation report that reports nothing is not evidence of quality; it is evidence of a weak process. Here is what these checks actually caught.

An independent code review found 15 defects. Two were critical:

What an audit of the patterns found (2026-09-16)

The documentation said every recognizer pattern and the RTF tokenizer stayed linear on pathological input, and listed the inputs: 5,000-character labels, 20,000 spaces, 50,000 control words. Three patterns were quadratic, and the corpus could not have shown it.

Pattern	Input	Before	After
<code>(?=[A-Z0-9-]*\d)[A-Z0-9]{2,}(?:-[A-Z0-9-]+)+</code>	64 KB of AB-AB-AB..., no digit	8,836 ms	1.9 ms
<code>[A-Za-z0-9._%+~]+@[A-Za-z0-9-]+\.[A-Za-z]{2,}</code>	64 KB of a.b-c_d..., no @	1,243 ms	3.7 ms
<code>[\t]+\n</code> in the RTF cleanup	64 KB of spaces, no newline	1,948 ms	0.02 ms

All three are the same shape: a quantified character class, then a literal that is not in the class. The pattern consumes the whole run, fails to find the literal, and starts again one character later -- `n` starting positions, each scanning $O(n)$.

The old corpus could not reach any of them. (`"AB-" * 4000`) + `"1"` ends in a digit, so the lookahead succeeded and the match completed at once. The email fixtures contained an `@`. The whitespace fixtures contained newlines. Every input allowed its pattern to finish, which is the one thing a backtracking test must not do. The budget was also wrong in kind: 5 seconds for four inputs passes a quadratic pattern comfortably.

The identifier lookahead was wrong a second way. `[A-Z0-9-]` includes the hyphen, so it scanned past a double hyphen to a digit the match can never reach -- `AB-Z--0911` was classified an identifier and the value returned was `AB-Z`, which contains no digit at all. The requirement now applies to the matched text, and agrees with the old pattern on every one of

20,024 compared strings except the twelve of that shape, where the old one was wrong.

Growth is now the assertion, in `verify_e2e.py` and in the unit suite: every regex literal in the package is run against long runs of the characters its own leading class accepts, at three sizes, and the check fails above 3x per doubling. A wall-clock budget moves with the machine; the ratio does not.

- RTF: `{*\generator}` -- a group Word, WordPad and RichEdit write into `*every*` file -- caused the rest of the document to be discarded. Essentially all real-world RTF returned an empty string. The existing unit test used a simpler construct and passed.
- HTML: `<meta>` and `<link>` have no closing tag but were treated as containers, which latched a counter open and dropped every byte after them. Every real web page, and every HTML-only email body, returned empty. The existing unit test used HTML with no `<meta>` tag.

Also found and fixed: script injection into the report; spreadsheet formula injection; an unbounded decompression path in the primary format; quadratic time and memory on malformed PDFs; one malformed character-map entry rendering an entire readable PDF undecodable; a field pattern that aborted a whole batch; a re-scan that overwrote its own audit trail; an archive that vanished without appearing in either output table; and a stray line in the test file that made running it directly execute 29 of the 64 then in the suite, and exit zero.

The end-to-end verifier then caught two more, including one in the documentation:

- A stated throughput of "50-70 pages per second" was wrong in both directions: the median is 101, but one image-heavy file runs at 42. The claim is now 40 or better, typically 100, and the check asserts the `*floor*`, because the floor is what you rely on.
- One of its own checks was too strict and failed on correct behaviour. It was corrected to test for a parsed tag rather than for escaped text.

Every one of those is now a named regression test.

8. What these results do not prove

- ****The datasheet records what the filesystem reported, which is not always the truth about the file.**** Timestamps can be set by any program with write access, and a foreign mount synthesizes ownership. The output flags what it cannot vouch for; it does not make the filesystem trustworthy.
- **SHA-256 in a workspace proves integrity, not authorship.** It detects drift. It is not a signature, and a workspace is not tamper-proof against someone who can write to it.
- **Benchmark corpora are not your documents.** Corpus A is software-generated, corpus B is commercial publishing. Neither is a folder of your invoices. Send a sample.
- **No OCR.** A scanned page holds an image, not text, and yields nothing -- reported, not silently blank.
- **Layout is approximated, not reconstructed.** Multi-column pages interleave. This is the main source of the gap against poppler on corpus B.
- **Fonts are keyed by resource name across a document**, not per page. A file reusing one font name for two different fonts on different pages can decode one of them wrong.
- **Encryption is detected, never bypassed.** This tool will not help you read a document you do not have the password for.
- ****176 passing checks means the documented claims hold today, on this machine, for these inputs.**** It does not mean the tool is free of defects. The review above found 15 after the unit tests were green.

9. Reproduce all of it

```
git clone https://github.com/brandonsbutler-ai/vanilla-extract
cd vanilla_extract
```

```
python3 -m unittest discover -s tests -v      # 104 unit tests
python3 verify_e2e.py                        # 176 end-to-end claim checks
python3 benchmark.py /path/to/your/pdfs      # quality against pdftotext
```

`benchmark.py` needs `poppler ( pdftotext )` for the comparison. Nothing else needs anything.
Every number in this document comes from those three commands.